

TUGAS 2
SISTEM OPERASI



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Dibuat Oleh :
Darina Qothrun Nada [2410651047]

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur: Membuat multiple child processes Setiap process melakukan tugas berbeda (sorting, searching, calculation) Parent process mengkoordinasi dan mengumpulkan hasil Tampilkan PID, execution time, dan status setiap proses.

Buat File manajemen_proses.c:

```
GNU nano 7.2 main_process_manager.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>

// Struktur untuk mengirim hasil dan waktu eksekusi dari child ke parent
typedef struct {
    pid_t pid;
    char task_name[20];
    double execution_time_ms;
    int exit_status;
    char result_info[100]; // Informasi hasil tambahan
} ProcessResult;

// --- Fungsi Tugas Proses Anak ---

// Tugas 1: Sorting (Contoh sederhana: Bubble Sort)
void task_sorting(int data[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (data[j] > data[j + 1]) {
                temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
            }
        }
    }
}

// Tugas 2: Searching (Contoh sederhana: Sequential Search)
int task_searching(int data[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (data[i] == target) {
            return i; // Ditemukan pada indeks
        }
    }
    return -1; // Tidak ditemukan
}

// Tugas 3: Calculation (Contoh sederhana: Penjumlahan deret)
long long task_calculation(int n) {
    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}

// --- Fungsi utama Child Process ---
void run_child_task(int task_id, int pipe_write_fd) {
    struct timeval start, end;
    ProcessResult result;
    gettimeofday(&start, NULL); // Catat waktu mulai
```

- Persiapan
 - Parent Process (PID 510) menyiapkan **Pipes** (saluran IPC) dan gettimeofday() untuk pengukuran waktu.
- Memuat Child

```

result.pid = getpid();
result.exit_status = EXIT_SUCCESS; // Asumsi berhasil

// Data sample untuk semua tugas
int data[] = {5, 1, 4, 2, 8};
int n = sizeof(data) / sizeof(data[0]);

if (task_id == 1) {
    strcpy(result.task_name, "Sorting");
    task_sorting(data, n);
    snprintf(result.result_info, sizeof(result.result_info), "Data terurut: %d, %d, %d, ...", data[0], data[1], data[2]);
} else if (task_id == 2) {
    strcpy(result.task_name, "Searching");
    int target = 4;
    int index = task_searching(data, n, target);
    if (index != -1) {
        snprintf(result.result_info, sizeof(result.result_info), "Angka %d ditemukan pada indeks %d.", target, index);
    } else {
        snprintf(result.result_info, sizeof(result.result_info), "Angka %d tidak ditemukan.", target);
    }
} else if (task_id == 3) {
    strcpy(result.task_name, "Calculation");
    int max_num = 10000;
    long long sum = task_calculation(max_num);
    snprintf(result.result_info, sizeof(result.result_info), "Penjumlahan 1 hingga %d: %lld", max_num, sum);
} else {
    strcpy(result.task_name, "Unknown Task");
    result.exit_status = EXIT_FAILURE;
    strcpy(result.result_info, "Tugas tidak dikenal.");
}

```

```

}

gettimeofday(&end, NULL); // Catat waktu selesai

// Hitung waktu eksekusi dalam milidetik
result.execution_time_ms = (end.tv_sec - start.tv_sec) * 1000.0;
result.execution_time_ms += (end.tv_usec - start.tv_usec) / 1000.0;

// Kirim hasil ke parent melalui pipe
write(pipe_write_fd, &result, sizeof(ProcessResult));
close(pipe_write_fd);

// Keluar dari proses anak
exit(result.exit_status);
}

// --- Fungsi utama Parent Process ---
int main() {
    printf("--- Sistem Manajemen Proses Sederhana ---\n");
    printf("PID Parent: %d\n", getpid());

    int num_children = 3;
    pid_t pids[num_children];
    int pipe_fds[num_children][2]; // Array untuk pipe [read_fd, write_fd]

    // 1. Membuat multiple child processes & Pipes
    for (int i = 0; i < num_children; i++) {
        if (pipe(pipe_fds[i]) == -1) {
            perror("pipe");

```

```

        exit(EXIT_FAILURE);
    }

    pids[i] = fork();

    if (pids[i] == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (pids[i] == 0) {
        // Kode Child Process
        close(pipe_fds[i][0]); // Tutup ujung baca (read end) di child
        run_child_task(i + 1, pipe_fds[i][1]); // Jalankan tugas
        // Program tidak akan mencapai sini karena run_child_task memanggil exit()
    } else {
        // Kode Parent Process
        close(pipe_fds[i][1]); // Tutup ujung tulis (write end) di parent
    }
}

// 2. Parent process mengkoordinasi dan mengumpulkan hasil
printf("Parent menunggu dan mengumpulkan hasil dari %d proses anak...\n", num_children);
printf("-----\n");
printf("| PID Proses | Nama Tugas | Waktu Eksekusi (ms) | Status Keluar | Hasil |");
printf("-----\n");

for (int i = 0; i < num_children; i++) {
    int status;
    pid_t terminated_pid = waitpid(pids[i], &status, 0); // Tunggu child spesifik

```

```
if (terminated_pid > 0) {
    ProcessResult result;
    ssize_t bytes_read = read(pipe_fds[i][0], &result, sizeof(ProcessResult)); // Baca hasil dari pipe

    if (bytes_read > 0) {
        // Tampilkan PID, execution time, dan status setiap proses
        char *status_str = "BERHASIL";
        if (WIFEXITED(status)) {
            if (WEXITSTATUS(status) != 0) {
                status_str = "GAGAL";
                result.exit_status = WEXITSTATUS(status);
            } else {
                result.exit_status = 0;
            }
        } else if (WIFSIGNALED(status)) {
            status_str = "TERMINASI SINYAL";
            result.exit_status = WTERMSIG(status);
        } else {
            status_str = "STATUS LAIN";
        }

        printf("| %-10d | %-11s | %-19.3f | %-13s | %-20s |\n",
            result.pid, result.task_name, result.execution_time_ms, status_str, result.result_info);
    } else {
        printf("| %-10d | (Tidak diketahui) | (Tidak diketahui) | ERROR BACA PIPE |\n", terminated_pid);
    }
    close(pipe_fds[i][0]); // Tutup ujung baca setelah selesai
} else if (terminated_pid == -1) {
    perror("waitpid");
}
}

printf("-----\n");
printf("Semua proses anak telah selesai. Parent process selesai.\n");
return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^N Replace ^P Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy

Compile dan Jalankan:

```
darin@DARINAQNADA:~$ gcc main_process_manager.c -o process_manager
darin@DARINAQNADA:~$ ./process_manager
--- Sistem Manajemen Proses Sederhana ---
PID Parent: 510

Parent menunggu dan mengumpulkan hasil dari 3 proses anak...
-----
| PID Proses | Nama Tugas | Waktu Eksekusi (ms) | Status Keluar | Hasil |
-----
| 511 | Sorting | 0.038 | BERHASIL | Data terurut: 1, 2, 4, ... |
| 512 | Searching | 0.042 | BERHASIL | Angka 4 ditemukan pada indeks 2. |
| 513 | Calculation | 0.084 | BERHASIL | Penjumlahan 1 hingga 10000: 50005000 |
-----
Semua proses anak telah selesai. Parent process selesai.
```

1. Persiapan

Fase ini berfokus pada inisialisasi dan deklarasi struktur data yang diperlukan untuk komunikasi dan pelaporan.

- **Struktur Data ProcessResult**

Struktur ini dideklarasikan untuk menstandarkan format data yang akan dikirim dari proses anak ke induk. Ini memastikan informasi krusial seperti PID, Nama Tugas, Waktu Eksekusi, Status Keluar, dan Informasi Hasil dapat dikirim dalam satu blok data yang terorganisir.

- **Deklarasi Tugas**

Tiga fungsi (`task_sorting`, `task_searching`, `task_calculation`) disiapkan. Ini mewakili beban kerja komputasi yang akan dieksekusi secara independen oleh proses anak.

- **Pengaturan di main()**

Proses induk mendefinisikan jumlah anak (`num_children = 3`), menyiapkan array untuk menyimpan PID anak (`pids`), dan array multidimensi untuk *file descriptor* Pipe (`pipe_fds[3][2]`). Setiap anak akan memiliki satu Pipe khusus untuk berkomunikasi dengan induk.

2. Fase Pembuatan Proses Anak (fork() dan Pipe)

Fase ini menggunakan perulangan untuk membuat setiap proses anak dan mengatur Pipe komunikasi spesifiknya.

1. **Pembuatan Pipe:** Di awal setiap iterasi, `pipe(pipe_fds[i])` dipanggil. Ini menciptakan sepasang *file descriptor* baru: `pipe_fds[i][0]` (ujung baca) dan `pipe_fds[i][1]` (ujung tulis).
2. **Duplikasi Proses (fork()):**
 - **Jika fork() mengembalikan 0 (Proses Anak):**
 - Anak hanya akan **menulis** hasil, sehingga ia menutup ujung baca Pipe: `close(pipe_fds[i][0])`.
 - Anak memanggil fungsi `run_child_task(i + 1, pipe_fds[i][1])`.
 - **Jika fork() mengembalikan PID (> 0) (Proses Induk):**
 - Induk hanya akan **membaca** hasil, sehingga ia menutup ujung tulis Pipe: `close(pipe_fds[i][1])`.
 - Induk menyimpan PID anak dan melanjutkan ke iterasi berikutnya atau fase berikutnya.

3. Fase Menjalankan Tugas Anak (run_child_task)

Setelah *forking*, setiap proses anak segera mengeksekusi fungsi `run_child_task` secara paralel.

1. **Pencatatan Waktu Mulai:** `gettimeofday(&start, NULL)` mencatat waktu sistem saat tugas dimulai.
2. **Eksekusi Tugas:** Tugas yang sesuai dengan `task_id` dijalankan (misalnya, `task_sorting`).
3. **Pencatatan Waktu Selesai:** Waktu akhir dicatat, dan waktu eksekusi (`execution_time_ms`) dihitung.
4. **Komunikasi Antar Proses (IPC):**
 - Proses anak menggunakan `write()` untuk mengirimkan seluruh blok data `ProcessResult` ke Pipe.
 - Anak menutup ujung tulis dan segera mengakhiri dirinya menggunakan `exit()`, yang juga menyampaikan kode status keluar (0 atau non-0) ke induk.

4. Fase Koordinasi Proses Induk

Proses induk menunggu setiap anak menyelesaikan eksekusi dan mengumpulkan hasilnya.

1. **Sinkronisasi**

Induk menggunakan `waitpid()` untuk menunggu anak spesifik (berdasarkan PID) menyelesaikan eksekusi. Ini penting untuk mendapatkan status keluar anak dan memastikan data siap dibaca.
2. **Membaca Hasil Pipe:**

- Setelah anak selesai, induk menggunakan `read()` untuk mengambil struktur `ProcessResult` dari ujung baca Pipe.
- **Analisis Status:** Makro status (`WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`) digunakan untuk mengonversi kode status numerik menjadi string yang mudah dibaca ("BERHASIL", "GAGAL", dll.).
- **Pelaporan dan Pembersihan:** Hasil akhir dicetak dalam bentuk tabel yang rapi. Terakhir, `close(pipe_fds[i][0])` dipanggil untuk membersihkan dan menutup ujung baca Pipe setelah selesai digunakan.

5. Fase Akhir Program

Setelah perulangan selesai, proses induk telah berhasil mengumpulkan status dan hasil dari semua proses anak. Proses induk mencetak pesan penyelesaian dan mengakhiri program dengan aman, membebaskan semua *resource* sistem yang dialokasikan.

Kesimpulan

Praktikum ini berhasil mengimplementasikan simulasi **Manajemen Proses** dan **Komunikasi Antar Proses (IPC)** menggunakan C di lingkungan Unix/Linux.

BAB II

CHAT APPLICATION DENGAN IPC

Implementasikan aplikasi chat sederhana menggunakan:

- Shared Memory atau Message Queue untuk menyimpan pesan
- Multiple processes sebagai pengguna
- Mutex/Semaphore untuk sinkronisasi
- Fitur: kirim pesan, lihat history, exit

```
darin@DARINAQNADA:~$ mkdir chat_aplication
darin@DARINAQNADA:~$ cd chat_aplication
darin@DARINAQNADA:~/chat_aplication$ nano chat_app.c
```

```
GNU nano 7.2 chat_app.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <sys/sem.h>

// Definisi kunci untuk Shared Memory dan Semaphore
#define SHM_KEY 1234
#define SEM_KEY 5678
#define MAX_MESSAGES 10 // Kapasitas riwayat pesan
#define MAX_MSG_LEN 100
#define USERNAME_LEN 20
#define NUM_USERS 2

// Struktur untuk satu pesan
typedef struct {
    char username[USERNAME_LEN];
    char text[MAX_MSG_LEN];
    time_t timestamp;
} Message;

// Struktur Shared Memory untuk riwayat chat
typedef struct {
    int count; // Jumlah pesan saat ini
    Message history[MAX_MESSAGES];
} ChatHistory;
```

```
// Union untuk inisialisasi semaphore (diperlukan di beberapa sistem)
union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
    struct seminfo *__buf;
};

// Fungsi untuk operasi P (Wait/Acquire) - Mengunci
void sem_wait(int sem_id) {
    struct sembuf sem_op;
    sem_op.sem_num = 0; // Hanya ada satu semaphore di set ini (indeks 0)
    sem_op.sem_op = -1; // Operasi P: mengurangi nilai semaphore
    sem_op.sem_flg = 0;
    semop(sem_id, &sem_op, 1);
}

// Fungsi untuk operasi V (Signal/Release) - Membuka Kunci
void sem_signal(int sem_id) {
    struct sembuf sem_op;
    sem_op.sem_num = 0;
    sem_op.sem_op = 1; // Operasi V: menambah nilai semaphore
    sem_op.sem_flg = 0;
    semop(sem_id, &sem_op, 1);
}

// Menampilkan seluruh riwayat pesan
void view_history(ChatHistory *chat_data, int sem_id) {
    sem_wait(sem_id); // Kunci Shared Memory sebelum membaca
```

```

printf("\n--- RIWAYAT CHAT ---\n");
if (chat_data->count == 0) {
    printf("[Kosong. Belum ada pesan dikirim.]\n");
} else {
    // Tampilkan pesan terbaru (dari 0 hingga count-1)
    for (int i = 0; i < chat_data->count; i++) {
        char time_buf[20];
        struct tm *tm_info = localtime(&chat_data->history[i].timestamp);
        strftime(time_buf, 20, "%H:%M:%S", tm_info);

        printf("[%s] %s: %s\n", time_buf,
            chat_data->history[i].username,
            chat_data->history[i].text);
    }
    printf("-----\n");

    sem_signal(sem_id); // Lepas kunci Shared Memory setelah selesai membaca
}

```

```

// Mengirim pesan baru
void send_message(ChatHistory *chat_data, int sem_id, const char *username) {
    char input_buffer[MAX_MSG_LEN + 1];

    printf("Masukkan pesan Anda (max %d karakter): ", MAX_MSG_LEN);
    // Memastikan baris input bersih sebelum membaca pesan
    if (fgets(input_buffer, sizeof(input_buffer), stdin) == NULL) return;

```

```

// Hapus newline jika ada
size_t len = strlen(input_buffer);
if (len > 0 && input_buffer[len - 1] == '\n') {
    input_buffer[len - 1] = '\0';
}

if (strlen(input_buffer) == 0) {
    printf("Pesan tidak boleh kosong.\n");
    return;
}

sem_wait(sem_id); // Kunci Shared Memory sebelum menulis

// Cek jika riwayat penuh, ganti pesan terlama (circular buffer)
int index_to_write = chat_data->count % MAX_MESSAGES;

// Copy data ke slot pesan
strncpy(chat_data->history[index_to_write].username, username, USERNAME_LEN);
chat_data->history[index_to_write].username[USERNAME_LEN - 1] = '\0';

strncpy(chat_data->history[index_to_write].text, input_buffer, MAX_MSG_LEN);
chat_data->history[index_to_write].text[MAX_MSG_LEN - 1] = '\0';

chat_data->history[index_to_write].timestamp = time(NULL);

// Tingkatkan hitungan pesan, tapi batasi agar tidak melebihi MAX_MESSAGES
if (chat_data->count < MAX_MESSAGES) {
    chat_data->count++;
} else {

```

```

// Jika sudah mencapai batas, count tetap di MAX_MESSAGES
// tapi index_to_write akan melakukan wrap-around
}

printf("\n[NOTIFIKASI] Pesan berhasil dikirim oleh %s.\n", username);

sem_signal(sem_id); // Lepas kunci Shared Memory
}

// Loop utama untuk setiap proses pengguna
void user_process(const char *username, ChatHistory *chat_data, int sem_id) {
    int choice;

    printf("--- Pengguna %s telah bergabung (PID: %d) ---\n", username, getpid());

    while (1) {
        printf("\n[%s] Pilih aksi:\n", username);
        printf("1. Kirim Pesan\n");
        printf("2. Lihat Riwayat Chat\n");
        printf("3. Keluar\n");
        printf("Pilihan Anda: ");

        // Membaca pilihan
        if (scanf("%d", &choice) != 1) {
            // Jika input tidak valid (bukan angka), bersihkan buffer
            while (getchar() != '\n');
            choice = 0;
        } else {
            // Bersihkan buffer setelah membaca angka

```

```

        while(getchar() != '\n');
    }

    switch (choice) {
        case 1:
            send_message(chat_data, sem_id, username);
            break;
        case 2:
            view_history(chat_data, sem_id);
            break;
        case 3:
            printf("\n--- %s keluar dari chat. ---\n", username);
            // Detach shared memory sebelum keluar
            if (shmdt(chat_data) == -1) {
                perror("shmdt");
            }
            exit(EXIT_SUCCESS);
        default:
            printf("[ERROR] Pilihan tidak valid. Silakan coba lagi.\n");
    }
}

int main() {
    int shm_id, sem_id;
    ChatHistory *chat_data = NULL;
    pid_t pids[NUM_USERS];

    // 1. SETUP SHARED MEMORY

```

```

    // shmget: Dapatkan ID Shared Memory
    shm_id = shmget(SHM_KEY, sizeof(ChatHistory), IPC_CREAT | 0666);
    if (shm_id == -1) {
        perror("shmget");
        return 1;
    }

    // shmat: Attach Shared Memory ke ruang alamat proses
    chat_data = (ChatHistory *)shmat(shm_id, NULL, 0);
    if (chat_data == (ChatHistory *)-1) {
        perror("shmat");
        return 1;
    }

    // Inisialisasi Shared Memory (hanya sekali oleh parent)
    memset(chat_data, 0, sizeof(ChatHistory));

    // 2. SETUP SEMAPHORE (Mutex)
    // semget: Dapatkan ID Semaphore set (hanya 1 semaphore)
    sem_id = semget(SEM_KEY, 1, IPC_CREAT | 0666);
    if (sem_id == -1) {
        perror("semget");
        // Lakukan cleanup Shared Memory
        shmctl(shm_id, IPC_RMID, NULL);
        return 1;
    }

    // Inisialisasi nilai Semaphore menjadi 1 (unlocked/available)
    union semun arg;

```

```

    arg.val = 1;
    if (semctl(sem_id, 0, SETVAL, arg) == -1) {
        perror("semctl SETVAL");
        // Lakukan cleanup
        shmctl(shm_id, IPC_RMID, NULL);
        semctl(sem_id, 0, IPC_RMID, NULL);
        return 1;
    }

    printf("Chat System Manager (PID: %d) Berjalan.\n", getpid());
    printf("Shared Memory dan Semaphore berhasil disiapkan.\n");
    printf("-----\n");

    // 3. FORK PROSES PENGGUNA
    const char *usernames[NUM_USERS] = {"Alice", "Bob"};

    for (int i = 0; i < NUM_USERS; i++) {
        pids[i] = fork();

        if (pids[i] == -1) {
            perror("fork");
            break;
        } else if (pids[i] == 0) {
            // Kode Proses Anak (Pengguna)
            user_process(usernames[i], chat_data, sem_id);
            // user_process memiliki exit(), jadi program tidak akan mencapai sini
            return 0;
        }
    }
}

```

```
// 4. KOORDINASI PROSES INDUK (Parent)
// Parent menunggu semua proses anak selesai
int status;
for (int i = 0; i < NUM_USERS; i++) {
    if (pids[i] > 0) {
        waitpid(pids[i], &status, 0);
        printf("[MANAGER] Pengguna %s (PID: %d) telah selesai.\n", usernames[i], pids[i]);
    }
}

// 5. CLEANUP DAN AKHIR PROGRAM
printf("\n-----\n");
printf("[MANAGER] Semua pengguna selesai. Membersihkan resource IPC...\n");

// Detach Shared Memory dari ruang alamat Parent
if (shmdt(chat_data) == -1) {
    perror("shmdt");
}

// Hapus Shared Memory dari sistem
if (shmctl(shm_id, IPC_RMID, NULL) == -1) {
    perror("shmctl IPC_RMID");
}

// Hapus Semaphore dari sistem
if (semctl(sem_id, 0, IPC_RMID, NULL) == -1) {
    perror("semctl IPC_RMID");
}
}
```

```
printf("[MANAGER] Resource IPC berhasil dihapus. Program selesai.\n");

return 0;
}
```

⌘G Help
⌘O Write Out
⌘W Where Is
⌘K Cut
⌘T Execute
⌘C Location
⌘U Undo
⌘A Set Mark
⌘X Exit
⌘R Read File
⌘N Replace
⌘U Paste
⌘J Justify
⌘/ Go To Line
⌘E Redo
⌘G Copy

```
darin@DARINAQNADA:~/chat_aplication$ gcc chat_app.c -o chat_app
darin@DARINAQNADA:~/chat_aplication$ ./chat_app
Chat System Manager (PID: 588) Berjalan.
Shared Memory dan Semaphore berhasil disiapkan.
-----
--- Pengguna Alice telah bergabung (PID: 589) ---

[Alice] Pilih aksi:
1. Kirim Pesan
2. Lihat Riwayat Chat
3. Keluar
Pilihan Anda: --- Pengguna Bob telah bergabung (PID: 590) ---

[Bob] Pilih aksi:
1. Kirim Pesan
2. Lihat Riwayat Chat
3. Keluar
Pilihan Anda: |
```

➤ Analisis dan Pembahasan Kode

Kode ini mengimplementasikan simulasi sistem chat dua pengguna (Alice dan Bob) yang berjalan sebagai proses terpisah dan berbagi riwayat pesan melalui Shared Memory yang dilindungi oleh Semaphore.

A. Struktur Data dan Definisi Kunci

- SHM_KEY dan SEM_KEY: Kunci unik yang digunakan untuk mengidentifikasi segmen Shared Memory dan set Semaphore.
- Message: Struktur untuk satu pesan, berisi username, text, dan timestamp.
- ChatHistory: Struktur utama di Shared Memory, berisi count (jumlah pesan saat ini) dan array history berkapasitas MAX_MESSAGES (10 pesan).

B. Fungsi Sinkronisasi (Semaphore)

- sem_wait(int sem_id): Mengimplementasikan operasi **P** (Lock/Kunci). Menggunakan semop dengan sem_op = -1. Proses akan menunggu di sini jika Shared Memory sedang dikunci oleh proses lain.

- `sem_signal(int sem_id)`: Mengimplementasikan operasi V (Unlock/Buka Kunci). Menggunakan semop dengan `sem_op = 1`. Melepaskan kunci sehingga proses lain dapat mengakses Shared Memory.

C. Fungsi Operasional Chat

1. `view_history(ChatHistory *chat_data, int sem_id)`

- Memanggil `sem_wait()` sebelum membaca riwayat untuk memastikan data yang dibaca konsisten dan tidak sedang dimodifikasi proses lain.
- Mencetak riwayat pesan yang ada.
- Memanggil `sem_signal()` setelah selesai membaca, melepaskan kunci.

2. `send_message(ChatHistory *chat_data, int sem_id, const char *username)`

- Meminta input pesan dari pengguna.
- Memanggil `sem_wait()` sebelum memodifikasi data.
- Menggunakan Circular Buffer (`index_to_write = chat_data->count % MAX_MESSAGES`) untuk menimpa pesan terlama jika kapasitas penuh (`MAX_MESSAGES`).
- Mengisi slot pesan dengan data baru (username, text, timestamp).
- Memperbarui `chat_data->count`.
- Memanggil `sem_signal()` setelah selesai menulis, melepaskan kunci.

D. Fungsi Utama (main dan user_process)

1. Setup Shared Memory (Parent Process)

- `shmget()`: Mendapatkan ID untuk Shared Memory sebesar `sizeof(ChatHistory)`.
- `shmat()`: Menghubungkan Shared Memory ke pointer `chat_data`.
- `memset()`: Menginisialisasi `chat_data` menjadi nol.

2. Setup Semaphore (Parent Process)

- `semget()`: Mendapatkan ID set Semaphore (1 semaphore).
- `semctl()`: Menginisialisasi nilai semaphore menjadi 1 (`arg.val = 1`), menandakan Shared Memory dalam keadaan terbuka (unlocked).

3. Fork Proses Pengguna (Parent Process)

- Loop for membuat `NUM_USERS (2)` proses anak menggunakan `fork()`.
- Di proses anak (`pids[i] == 0`), fungsi `user_process()` dipanggil, memulai interaksi chat.
- Di proses induk (`pids[i] > 0`), ID proses anak disimpan.

4. **user_process() (Child Process)**

- Berisi loop interaksi menu (Kirim Pesan, Lihat Riwayat, Keluar).
- Akses ke fungsi `send_message` dan `view_history` secara otomatis melibatkan sinkronisasi Semaphore.
- Saat keluar, memanggil `shmdt()` untuk melepaskan Shared Memory.

5. **Cleanup (Parent Process)**

- Parent menunggu semua anak selesai menggunakan `waitpid()`.
- Setelah semua anak selesai, Parent melakukan pembersihan:
 - o `shmdt()`: Melepaskan Shared Memory dari parent.
 - o `shmctl(..., IPC_RMID, ...)`: Menghapus Shared Memory dari sistem.
 - o `semctl(..., IPC_RMID, ...)`: Menghapus Semaphore dari sistem.

Kesimpulan

Program ini berhasil mendemonstrasikan implementasi Inter-Process Communication (IPC) menggunakan Shared Memory dan Semaphore. Shared Memory digunakan untuk menyimpan riwayat chat yang dapat diakses oleh proses-proses pengguna (Alice dan Bob) secara efisien. Semaphore berfungsi sebagai mutex yang menjamin Mutual Exclusion, yaitu hanya satu proses yang dapat menulis atau membaca data Shared Memory pada satu waktu, sehingga mencegah race condition dan menjaga integritas data riwayat chat. Penggunaan `fork()` memungkinkan simulasi aplikasi multi-proses yang saling berinteraksi dan berbagi sumber daya.